

BACKEND DEVELOPER PORTFOLIO

서영석

기능 구현에서 멈추지 않고, 문제를 분석하고 개선 결과를 수치로 검증하는 백엔드 개발자를 지향합니다.

GITHUB

github.com/yss9

EMAIL

yse2196@gmail.com

Profile



기본 정보

이름	서영석
Email	yse2196@gmail.com
GitHub	github.com/yss9
학력	영남대학교 컴퓨터공학과 졸업
전공 평점	3.84 / 4.5
자격증	정보처리기사
교육	삼성청년SW/AI 아카데미(2026/01 ~)

저는 새로운 기능을 빠르게 만드는 것보다, 기존 서비스의 구조와 흐름을 이해하고 안정적으로 개선하는 과정을 중요하게 생각합니다.

프로젝트에서는 사용자 흐름과 데이터 처리 과정을 먼저 파악하고, 백엔드 API와 데이터베이스가 자연스럽게 연결되도록 구현해왔습니다.

문제가 발생하면 로그, 실행 순서, 쿼리와 데이터 흐름을 따라 원인을 좁히고, 성능 개선 전후를 수치로 검증하며 서비스의 안정성과 완성도를 높여왔습니다.



목적 중심의 기능 구현

단순한 코드 작성을 넘어, 서비스의 목적과 실제 사용자 흐름 고려하여 서비스를 개발합니다.



데이터 기반 문제 해결

추측이 아닌 로그, 실행 순서, 데이터 흐름을 추적하여 문제의 원인을 논리적으로 좁혀나갑니다.



수치로 증명하는 성능

JMeter를 활용하여 병목 유형을 분석하고, Before/After 수치를 통해 성능 개선 성과를 검증합니다.

Tech Stack

Backend

Java Spring Boot Spring Security
JPA QueryDSL

REST API 개발, 사용자 인증/인가, 파일 업로드 처리, 동적 쿼리를 활용한 복잡한 검색 로직 구현에 활용합니다.

Database

MySQL PostgreSQL SQLite

JPA 기반 데이터 저장/조회와 AWS RDS 연동 경험이 있습니다. 인덱스·쿼리 튜닝은 학습과 테스트 범위에서 검증하며 적용했습니다.

Realtime

WebSocket STOMP WebRTC
Redis Pub/Sub

실시간 채팅, WebRTC 시그널링, 메시지 중계 구조를 프로젝트 범위 안에서 설계하고 구현했습니다.

Infra / Cloud

EC2 / S3 / RDS CloudFront Docker
GitHub Actions

컨테이너 기반 배포, HTTPS 보안 설정(ALB/ACM), 정적/동적 리소스 분리 및 CI/CD 파이프라인을 구축합니다.

Testing / Tools

Git / GitHub JMeter IntelliJ

Git/GitHub 기반 버전 관리와 JMeter를 활용한 병목 구간의 Before/After 수치 비교를 수행했습니다.

Frontend

React JavaScript CSS

백엔드 API와 연결되는 사용자 흐름을 이해하고, 프로젝트에 필요한 React 화면을 구현했습니다.

실제 프로젝트에서 어떤 목적과 흐름에 사용했는지 기준으로 정리했습니다.

Project Map

01

SayBridge 1인 개발 FE/BE · 화상채팅 기반 외국어 교육 플랫폼

React, Spring Boot, Spring Security, JWT, OAuth2, Spring Data JPA, QueryDSL, MySQL, Redis, WebSocket, WebRTC, AWS S3

🔧 WebRTC 검은 화면 해결 (시그널링 순서 제어) / 과제 제출 상태 N+1 쿼리 성능 개선

02

Glople 5인 개발(FE/BE) + 2인 디자인 팀장 · 추천 로직 구현 · MBTI 기반 여행지 추천 커뮤니티

React, Spring Boot, Spring Security, JWT, Spring Data JPA, MySQL, WebSocket

📈 추천 로직 병목 분석 (문자열 탐색 vs 전체 정렬) / Bitmask 및 Top-K 방식으로 테스트랩 성능 개선

03

BlueMemories 1인 개발 + 1인 디자인 · AI 감정 기록 및 멘탈케어 커뮤니티

React, Spring Boot, Spring Security, JWT, Spring Data JPA, MySQL, AWS S3

🤖 GPT API 응답 파싱 오류 해결 / 공개 일기 목록 JOIN Projection 최적화

04

AWS Deploy 1인 구축 · 풀스택 배포 자동화 파이프라인

Docker, GitHub Actions, AWS S3, CloudFront, Route 53, ALB, EC2, RDS, ACM

☁️ Mixed Content 해결 (HTTPS 구성) / RDS 연결 오류 및 CI/CD SSH Timeout 트러블슈팅

Overview

<https://github.com/yss9/SayBridge>

SayBridge 화상채팅 기반 외국어 교육 플랫폼

관심사에 맞는 수업을 선택하고 선생님과 1:1 화상 수업을 진행하는 서비스입니다. 수업 개설부터 화상 통화, 실시간 채팅, 과제 제출까지 전체 학습 흐름을 통합 구현했습니다.

기간 2025.01 ~ 2025.05

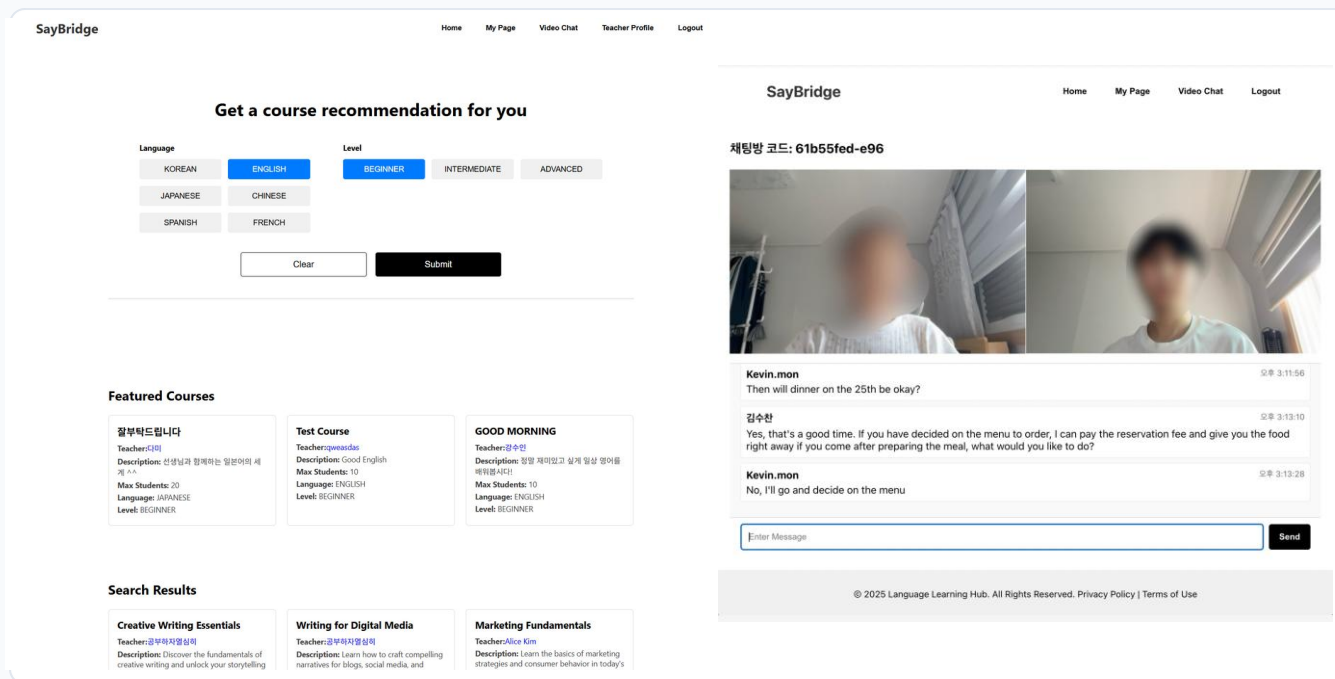
인원 1인 개발

역할 프론트엔드 및 백엔드 개발

Spring Boot React MySQL Redis

기술 스택 WebRTC WebSocket Security/JWT

AWS S3



🛡️ 인증 및 권한 관리

Spring Security/OAuth2/JWT
기반 소셜 로그인 및 역할별 접근 제어

🗣️ WebRTC 화상 연결

브라우저 간 미디어 스트림 P2P 연결을 통한 실시간 화상 수업 구현

📩 실시간 메시징

WebSocket/STOMP 기반의 시그널링 처리 및 수업 중 실시간 채팅 중계

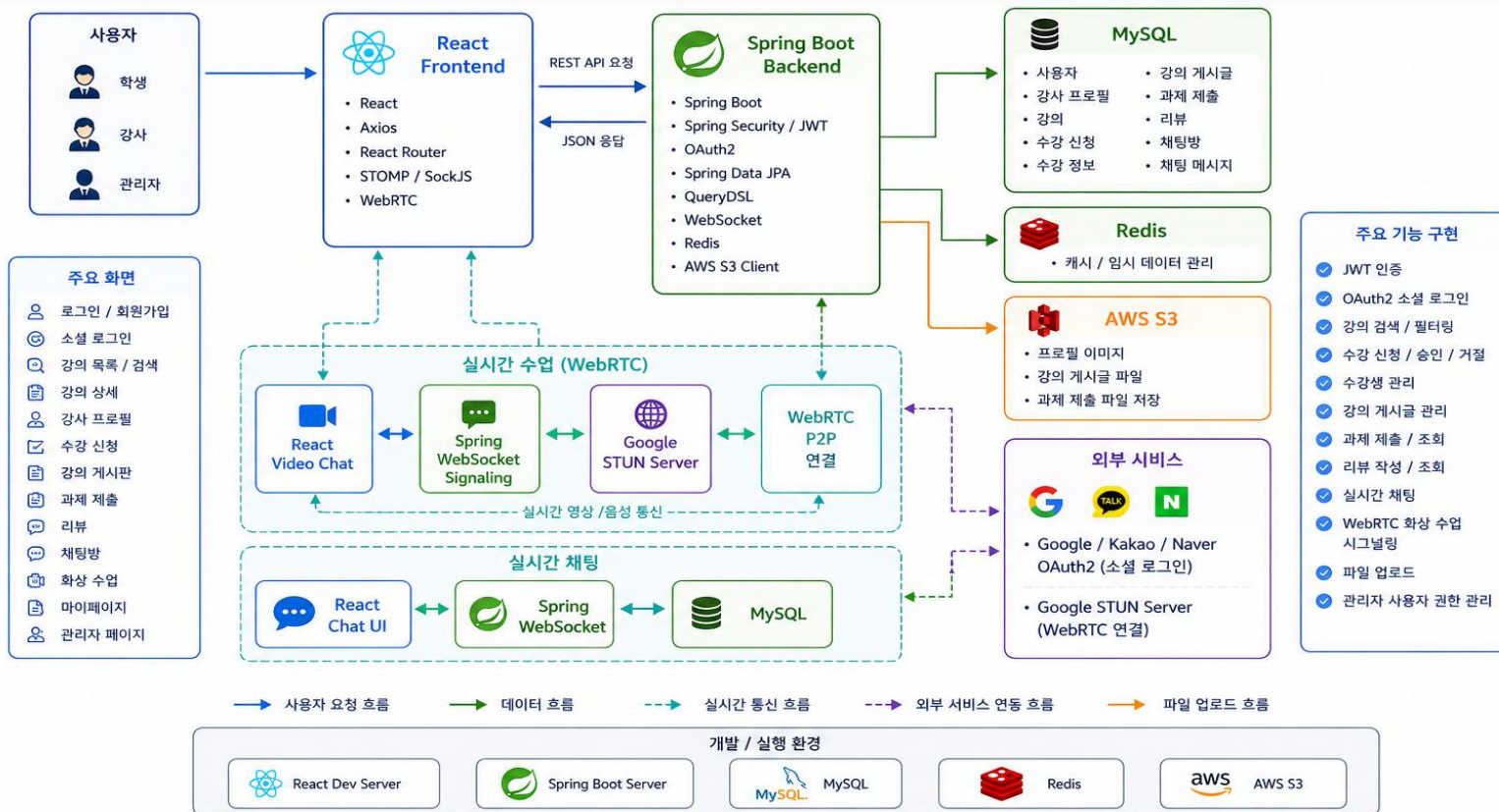
+ 동적 강의 검색

QueryDSL을 활용하여 언어·레벨 등 복합 조건의 동적 강의 검색 구현

☁️ 파일 시스템 분리

S3를 활용한 강의 자료/과제 파일 저장 및 과제 제출 상태 조회 구조 개선

System Architecture



React

수업 탐색, 신청, 화상 수업, 채팅, 과제 제출 화면 및 클라이언트 로직 담당

Spring Boot

인증(JWT), 강의/과제 CRUD, 파일 업로드 REST API 및 비즈니스 로직 처리

WebSocket/STOMP

실시간 채팅 메시지 중계 및 WebRTC 연결을 위한 시그널링 데이터 교환

WebRTC

서버 부하를 최소화하기 위한 브라우저 간(P2P) 영상·음성 미디어 스트림 연결

Storage Group

영속 데이터(MySQL), 파일(S3), 메시지 중계 확장성(Redis)을 고려한 데이터 설계

Troubleshooting — WebRTC 검은 화면 해결

❗ 문제 Offer/Answer 교환은 정상이었지만 상대방 영상이 표시되지 않는 검은 화면 발생 — SDP 교환 성공 여부만으로는 미디어 스트림 연결 상태를 보장할 수 없음

1 문제 현상

WebSocket 시그널링 로그상 Offer/Answer 교환 정상 완료. 그러나 양측 화면 모두 검은 화면 유지, 미디어 스트림 수신 없음

2 원인 추적

브라우저 콘솔에서 SDP를 직접 복사해 상대방에게 수동 주입하는 테스트 진행. WebRTC 코어 연결 문제와 자동화 시그널링 로직 문제를 분리해 범위 축소

3 실제 원인

SDP 로그 비교 결과, addTrack() 완료 전에 createOffer()가 먼저 실행됨. 자동 생성 SDP에서 m-line 누락 및 direction 비활성 상태 확인

4 해결

async/await로 미디어 트랙 등록 완료를 보장한 이후 Offer SDP를 생성하도록 실행 순서 제어

5 결과

영상 연결 정상화. SDP 생성 시점과 PeerConnection 상태를 함께 확인해야 하며, 시그널링 성공이 곧 미디어 연결 성공을 의미하지 않음을 학습

● ● ● webrtc-signaling.js — 핵심 수정 코드

```
// 1) 먼저 로컬 미디어를 확보하고 PeerConnection에 등록
navigator.mediaDevices
  .getUserMedia({ video: true, audio: true })
  .then((stream) => {
    localRef.current.srcObject = stream;
    stream.getTracks().forEach((track) => {
      if (pc.signalingState !== 'closed') {
        pc.addTrack(track, stream);
      }
    });
  })
  .then(connectStomp);

// 2) 상대방 입장(join) 후 stable 상태에서 Offer 생성
case 'join':
  if (isInitiator && !negotiatingRef.current &&
    pc.signalingState === 'stable') {
    negotiatingRef.current = true;
    const offer = await pc.createOffer();
    await pc.setLocalDescription(offer);
    stompRef.current.publish({
      destination: '/app/video.signal',
      body: JSON.stringify({
        type: 'offer',
        sdp: pc.localDescription,
        chatCode,
      }),
    });
  }
  break;
```

Performance Improvement

 게시글 목록에서 학생의 과제 제출 상태를 표시하기 위해 게시글마다 단건 조회가 반복되는 N+1 패턴을 발견했습니다. 이를 IN 조건 배치 조회와 Projection으로 최적화하여 DB 부하를 줄이고 응답 속도를 개선했습니다.

⚠ Before: 반복 쿼리 (N+1)

```
for (postId : postIds) {  
    submission = findSubmission(postId, user);  
    if (submission != null) {  
        result.put(postId, submission.url);  
    }  
}
```

✅ After: IN 조건 배치 조회

```
SELECT postId, attachmentUrl  
FROM Homework  
WHERE studentId = :studentId  
AND postId IN :coursePostIds
```

DB Query / Request

100→1

Avg Response Time

16.75ms→3.99ms

p95 Latency

20ms→5ms

Error Rate

0%

 측정 도구: JMeter 조건: 30 threads × 20 loops (Total 600)

Overview

Glople

MBTI 기반 여행지 추천

설명 사용자 성향(MBTI, 연령, 성별)을 기반으로 맞춤형 여행지와 루트를 제안하는 커뮤니티 서비스

기간 2024.03 ~ 2024.11

인원/역할 5인 / 팀장, 추천 로직 구현 및 프론트 일부 개발

기술 스택 Spring Boot React MySQL Spring Security

WebSocket

외부 API OpenAI API Google Maps API Google Places API

Google STT API Wikipedia API RestCountries API



01 개인 맞춤형 추천

MBTI 4축과 인구통계 정보를 벡터화하여 코사인 유사도 기반 맞춤형 루트 추천

02 키워드 필터링

사용자가 선택한 관심 키워드를 여행지 설명과 매칭해 관련 여행지를 정렬

03 AI 대화형 챗봇

OpenAI API 연동을 통한 자연어 기반 여행지 질문 및 추천 답변 생성

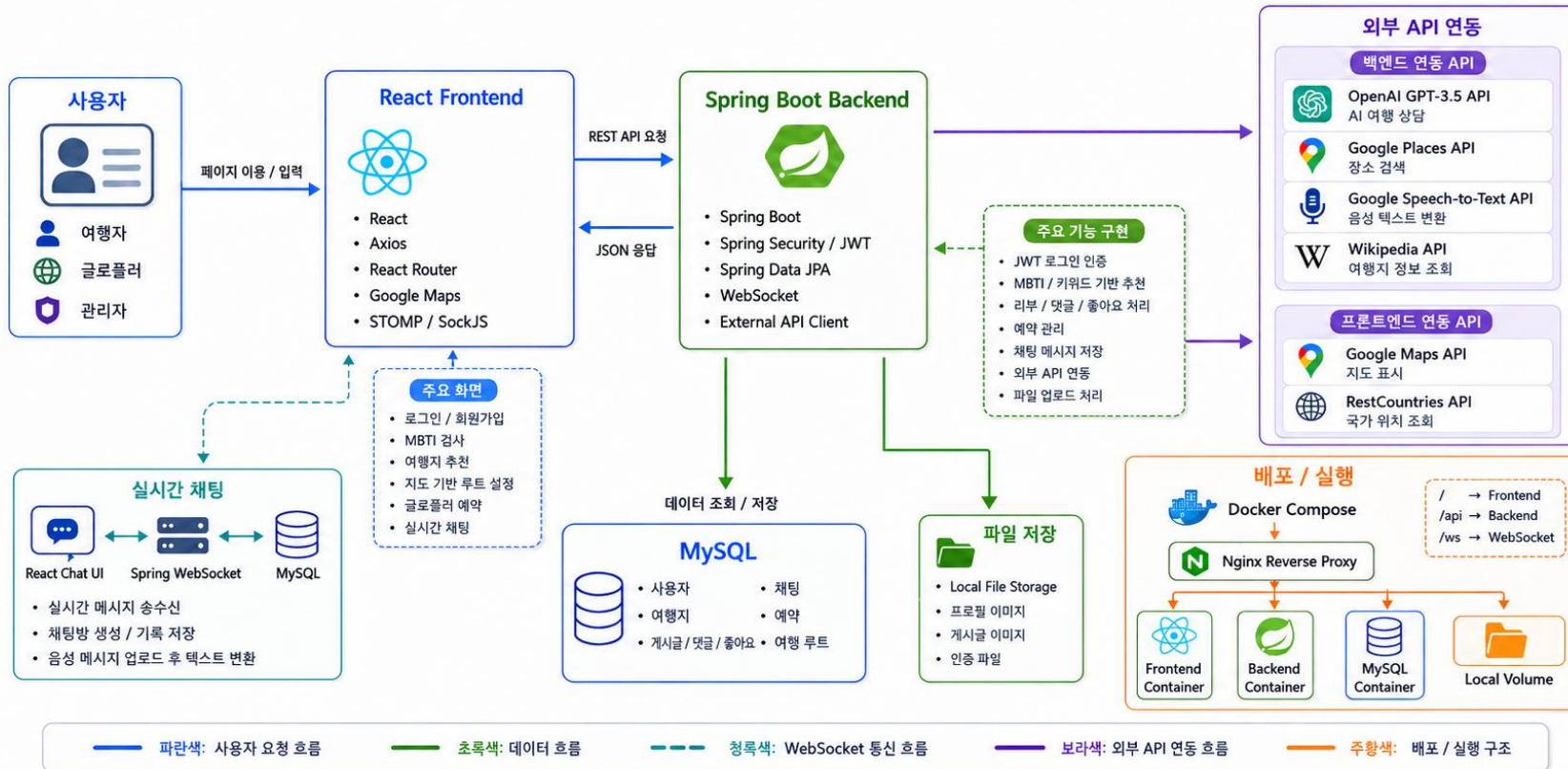
04 실시간 매칭/채팅

WebSocket을 활용하여 사용자 간 매칭 및 채팅 흐름 구현

05 팀 리딩 및 관리

5인 규모 팀의 일정 관리, 기능 조율, 발표 준비 등 프로젝트 진행 총괄

System Architecture



React

여행지 추천 입력 인터페이스, 결과 시각화, 커뮤니티 UI 전반 담당

Spring Boot

추천 알고리즘 처리, 외부 API 연동 제어, REST API 및 실시간 통신 중계

MySQL

사용자 성향(MBTI 등), 여행지 메타데이터, 사용자 루트 및 후기 데이터 관리

External APIs

OpenAI(자연어 추천), Google Places/Wikipedia(여행 정보 보강) 데이터 수집

WebSocket

사용자 간의 실시간 매칭 상태 공유 및 채팅 메시지 송수신 흐름 처리

Analysis

🎯 1) 추천 정확도 문제 (Accuracy)

초기 설계는 사용자를 16가지 MBTI 유형 중 하나로 분류했습니다. 동일 MBTI 안의 취향 차이와 세부 성향 유사도를 충분히 반영하기 어려웠습니다.

→ 개선 방향: MBTI 4축 + 인구통계 벡터화 및 코사인 유사도 도입

🔍 2) Keyword 추천 병목 (String Search)

전체 데이터를 조회(findAll)한 뒤 Java에서 문자열 탐색(contains)을 반복했습니다. 정렬 과정에서도 매칭 키워드 수 계산이 중복되었습니다.

→ 개선 방향: keyword_mask 후보 필터링 + Java bitCount 점수 계산

▼ 3) MBTI 추천 병목 (Sorting Cost)

상위 12건만 필요하지만 전체 사용자에게 대해 벡터 변환과 유사도 계산을 수행한 뒤 전체 정렬 후 limit을 적용했습니다.

→ 개선 방향: 성별/연령 후보군 축소 + PriorityQueue Top-K 추출

Performance Improvement

 Before: 전체 조회 및 Java 레벨 처리

Keyword 추천 (문자열 탐색 비용)

```
findAll()  
  .filter(containsKeyword)  
  .sorted(countMatch desc)  
  .limit(21)
```

MBTI 추천 (전체 계산 및 정렬)

```
findAllUsers()  
  .map(cosineSimilarity)  
  .sorted(desc)  
  .limit(12)
```

Keyword 추천 Avg

15.7ms **8.92ms**

▼ 43.3% 감소

Keyword 추천 p95

20ms **10ms**

▼ 50.0% 감소

 After: DB 비트 연산 및 Top-K 알고리즘

Bitmask 필터링 도입

```
WHERE (keyword_mask & :mask) <> 0  
score = bitCount(keyword_mask & mask)
```

후보군 축소 및 PriorityQueue

```
candidates = findByGenderAndBirthYear()  
topScores = PriorityQueue(12)
```

MBTI 추천 Avg

27.9ms **7.21ms**

▼ 74.2% 감소

MBTI 추천 p95

33ms **9ms**

▼ 72.7% 감소

Overview

<https://github.com/yss9/BlueMemories>

BlueMemories

AI 기반 감정 기록 및 멘탈케어 커뮤니티

상세 내용 기존 서비스 리뉴얼을 통해 감정 분석 및 콘텐츠 추천, 공유 일기장 구조를 확장한 멘탈케어 서비스입니다.

기간/인원 2024.07 ~ 2024.09 (2인: 개발 1, 디자인 1)

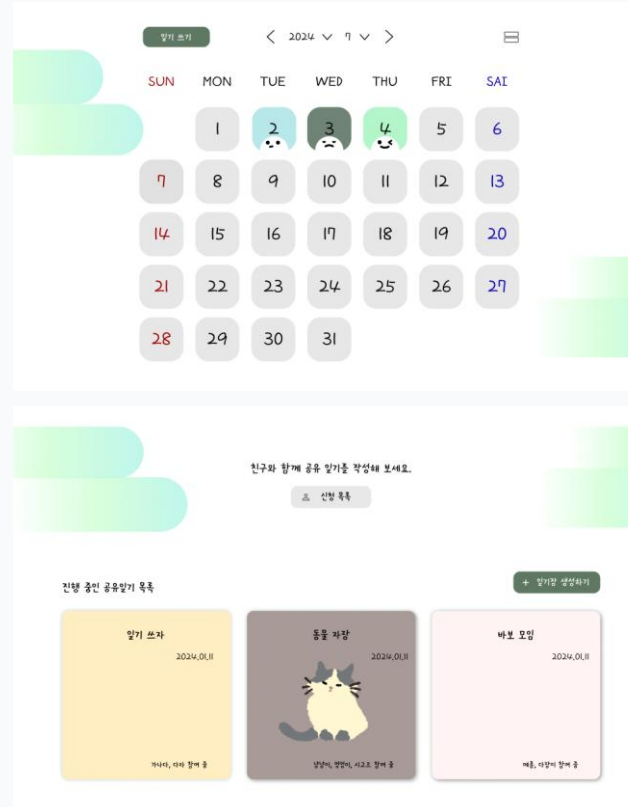
나의 역할 기획 및 백엔드/프론트엔드 개발 총괄

기술 스택

- Spring Boot
- React
- MySQL
- Spring MVC
- WebClient
- WebSocket
- Security/JWT
- AWS S3

외부 API

- Naver Clova Sentiment API
- OpenAI API
- Youtube Data API



일기 CRUD 구현

일기 작성, 조회, 수정, 삭제를 위한 안정적인 REST API 및 사용자 인터페이스 구현

AI 감정 분석 연동

Naver Sentiment API를 활용하여 작성된 일기 텍스트의 감정 수치를 분석하고 저장

맞춤형 콘텐츠 추천

OpenAI와 YouTube API를 조합하여 분석된 감정에 어울리는 음악 및 영상 추천 로직 설계

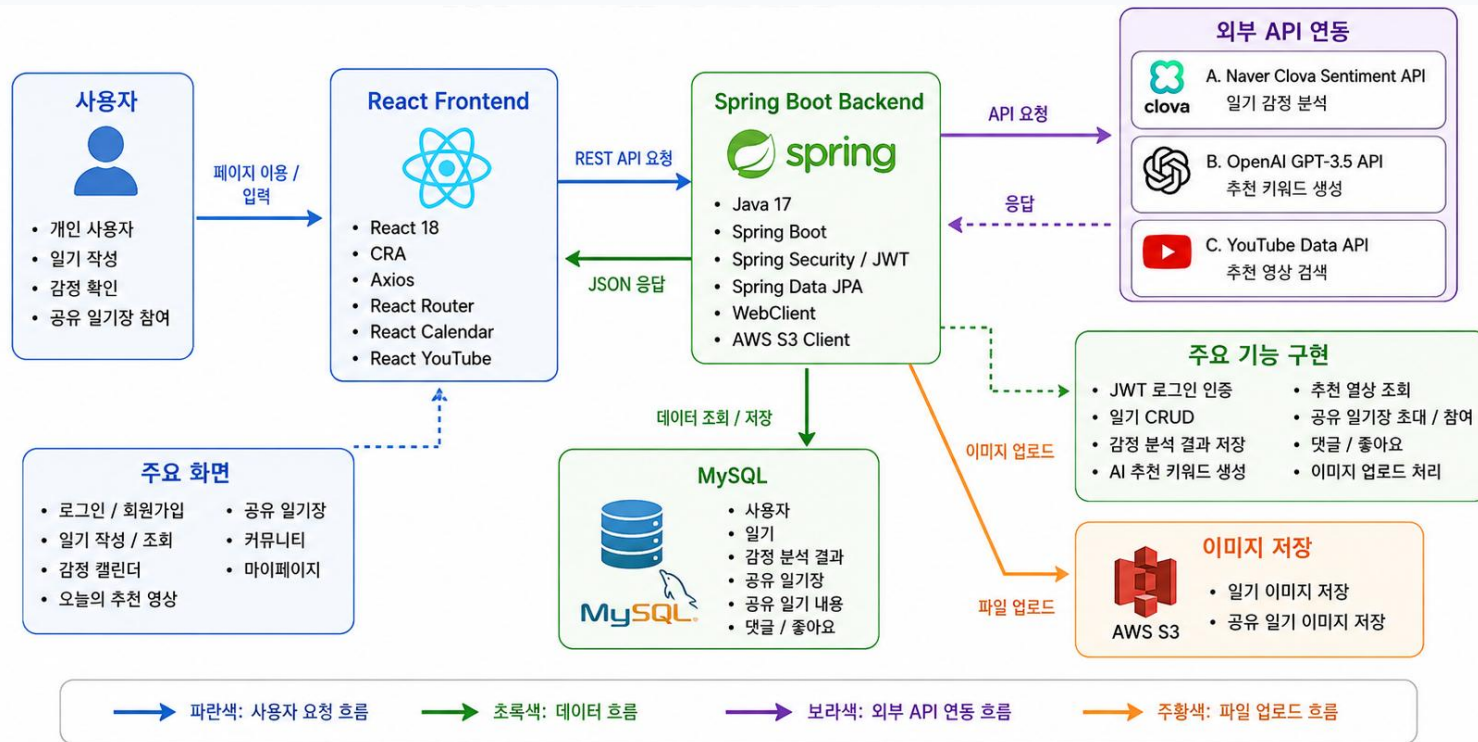
공유 일기장 확장

기존 1:1 교환일기 방식에서 N:M 그룹 공유 일기장 구조로 데이터 스키마 및 기능 확장

인증 및 파일 저장

JWT 기반 보안 인증 체계 구축 및 AWS S3를 연동한 이미지 업로드/관리 구조 최적화

System Architecture



React

일기 작성, 감정 캘린더, 커뮤니티 화면 및 사용자 인터렉션 담당

Spring Boot

일기, 공유 일기장, 댓글, 공감, 추천 로직 등 핵심 API 비즈니스 로직 처리

MySQL

사용자 프로필, 일기 데이터, 감정 분석 결과, 커뮤니티 관계망 저장

AI APIs

Naver(감정 분석), OpenAI(키워드 생성), YouTube(콘텐츠 검색) 연동

AWS S3

일기 내 첨부 이미지 및 정적 리소스를 서버와 분리하여 효율적으로 관리

BlueMemories | Troubleshooting

정규식으로 GPT 응답을 억지로 파싱하던 구조를 JSON 계약 + DTO 검증 + fallback 구조로 바꿔 추천 실패를 줄였습니다.

AS-IS

정규식 기반 파싱 구조

프롬프트로 번호 형식을 유도하고 문자열에서 추천 키워드를 추출

문제

응답 순서·문구가 조금만 바뀌어도 키워드 추출 실패

원인 / 실제 구현

- “1. 노래 / 3. 검색어” 같은 번호 형식에 의존
- Pattern 정규식으로 타입과 내용을 분리
- GPT가 표현을 섞거나 순서를 바꾸면 누락 발생

한계

고정 형식에서는 동작하지만 실제 응답 변형에 취약

**TO-BE**

JSON 계약 + DTO 검증 구조

GPT 응답을 서버가 검증 가능한 데이터 구조로 고정

해결 1

songs[], searchKeywords[] 형태의 JSON 응답으로 고정

해결 2

- ObjectMapper로 RecommendationResponseDto 파싱
- songs 2개, searchKeywords 2개 필수 조건 검증
- 필드 누락·개수 불일치를 서버에서 감지

예외 처리

파싱 실패 시 기본 추천값을 반환해 사용자 흐름 유지

결과 응답 형식이 흔들려도 추천 실패를 감지하고, 검증된 데이터만 저장하도록 AI 추천 처리 흐름의 안정성을 높였습니다.

Performance Improvement: 공개 일기 목록 조회 최적화

🔔 Before: 비효율적 전체 조회

- ❌ 공개 일기 전체 조회: 데이터 규모와 관계없이 모든 레코드를 메모리에 로드
- ❌ Entity 순회 변환: Diary 엔티티와 연관된 User 정보를 개별 순회하며 DTO로 변환하여 지연 로딩 발생
- ❌ 과도한 Payload: 목록 화면에서 불필요한 일기 본문, 전체 이미지 경로 등 큰 데이터가 모두 포함되어 전송

🔗 After: Pagination & JOIN Projection

- ✅ Page 기반 최신순 조회: 필요한 만큼만 데이터를 끊어서 가져오도록 Pagination 적용
- ✅ JOIN Projection: DTO 전용 쿼리를 사용하여 목록 화면에 필수적인 필드(제목, 작성자명 등)만 선별 조회
- ✅ 응답 크기 최적화: 대용량 텍스트 필드를 제외하고 페이지 단위 응답으로 응답 범위와 크기를 대폭 축소

DB QUERY / REQUEST

1001 → 1

-99.9%

RESPONSE PAYLOAD SIZE

748KB → 4KB

-99.4%

AVG RESPONSE TIME

147.85ms → 3.63ms

-97.5%

P95 RESPONSE TIME

161.95ms → 5ms

-96.9%

🏗️ 측정 조건: 사용자 1,000명 / 일기 2,000건 / JMeter 20 threads × 15 loops (300 requests)

AWS 기반 풀스택 배포 자동화 파이프라인 구축

<https://github.com/yss9/todo-list-server>
<https://github.com/yss9/todo-list-client>

AWS Deploy

풀스택 애플리케이션 배포 자동화

백엔드는 Docker 컨테이너화하여 EC2에서 실행하고, 프론트엔드는 S3와 CloudFront로 정적 배포를 수행했습니다. GitHub Actions를 통해 전체 배포 파이프라인을 자동화했습니다.

기간 2025.09 (1인 구축)

역할 인프라 설계 및 배포 자동화 구축

기술 스택

React	Node.js	Docker	Github Actions	AWS S3	CloudFront	Route 53
ALB	EC2	RDS	ACM			

Frontend

React Build -> S3 정적 호스팅 -> CloudFront CDN 배포

Backend

GitHub Actions -> Docker Image -> EC2 Container 배포

Database

EC2 서버와 RDS PostgreSQL 연동

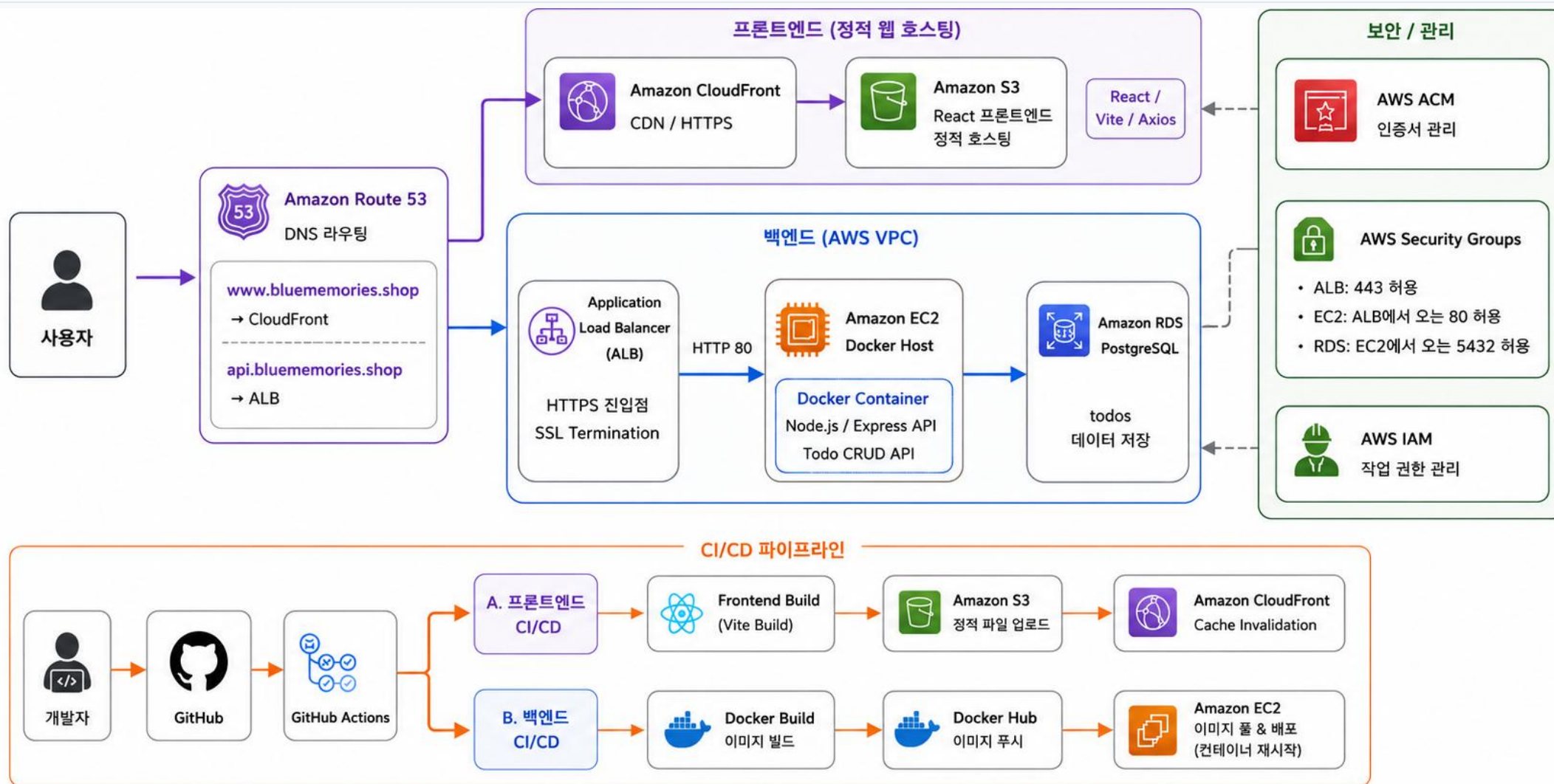
HTTPS

CloudFront/ALB + ACM 활용, 외부 요청 구간 HTTPS 적용

DNS

Route53 도메인 연결 및 ACM 인증서 적용

System Architecture



Troubleshooting

01 Mixed Content HTTPS 페이지에서 HTTP API 요청이 차단되어 통신 불가

NETWORK

원인 CloudFront HTTPS → EC2 HTTP API 호출
Frontend/Backend 프로토콜 불일치

→ **해결** ALB 구성 + ACM 인증서 연결
Backend API Endpoint HTTPS 통일

✓ 프론트엔드와 백엔드 간 통신 프로토콜의 일관성과 보안 계층 설계의 중요성을 확인

02 500 Internal Server Error 배포 직후 서버는 정상 실행됐지만 모든 API 요청에서 500 오류 발생

DATABASE

원인 RDS Security Group 미설정 · Secrets 값 오류
DB URL 생성 · SSL 옵션 불일치

→ **해결** docker logs로 서버 오류 추적 → RDS 접근 허용
Secrets 검증 → DB 연결 정보 및 SSL 옵션 수정

✓ 로그를 기준으로 네트워크 → 인증 → DB 연결 순서로 원인의 범위를 좁혀 해결

03 CI/CD SSH Timeout GitHub Actions 배포 단계에서 EC2 SSH 연결 시간 초과 발생

CI/CD

원인 GitHub Actions Runner 공인 IP 변경
EC2 Security Group 특정 IP 허용으로 접근 차단

→ **해결** 배포 서버 접근 정책 재검토 · 고정 접근 환경 검토
AWS SSM 기반 배포 방식 고려

✓ Runner IP가 고정되지 않는 환경에서는 네트워크 변동성을 고려한 배포 구조가 필요함

+ 문제 해결 방식

브라우저 오류



네트워크 경로



로드밸런서/인증서



서버 로그



DB 연결

추적으로 설정을 변경하기보다 실행 경로와 로그를 기준으로 원인의 범위를 단계적으로 좁혀 확인했습니다.